

Academic Event & Resource Booking Portal

Full-Stack MERN + FastAPI Concurrency Engine, Security Architecture & Comprehensive Viva Q&A

Target Domain: Centralized Campus Resource & Facility Scheduling

Core Invariant: Structurally Impossible Double-Booking Guaranteed

Architecture: Layered MVC + Microservice Sidecar + Serverless Edge

Security: Dual-Token JWT + RBAC + Stateless HMAC Verification

Deployment: Vercel Edge SPA + Serverless Express + Cloud Atlas DB

Prepared for Technical Review, Architecture Defense, and Company Tech Interviews

Author / Lead Developer: Keerthana (Easwari Engineering College)

Version: 2.4.0 Production Release • **Date:** August 2026

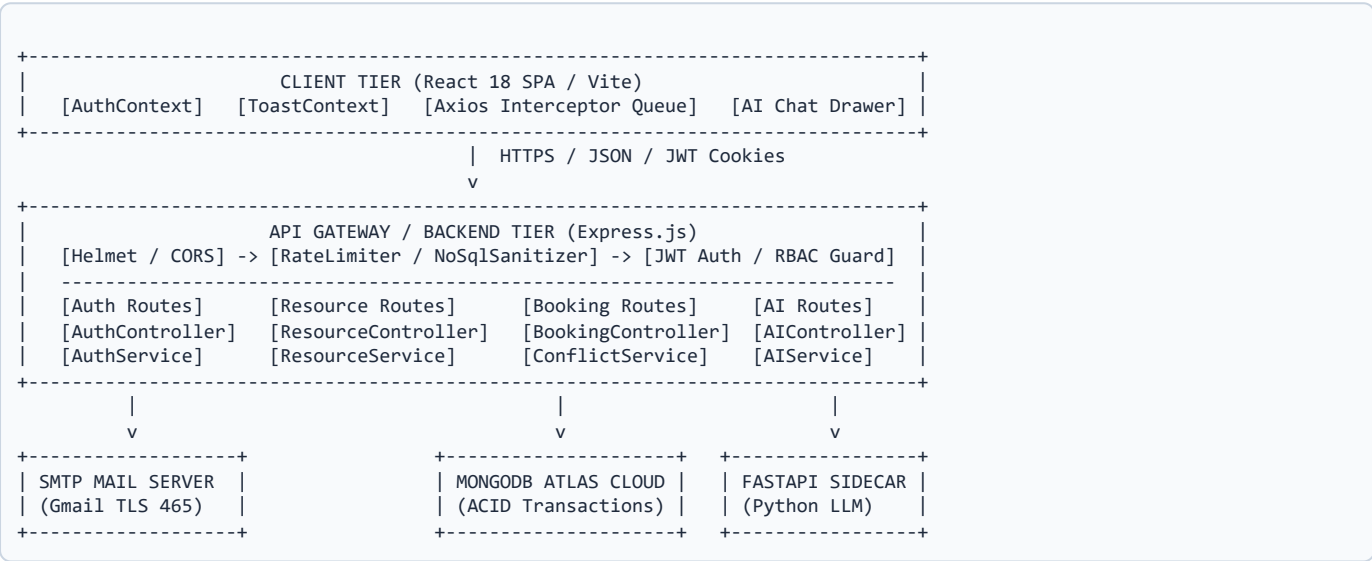
1. Executive Summary & Problem Domain

The **Academic Event & Resource Booking Portal** is an enterprise-grade platform engineered to eliminate scheduling chaos across academic institutions. It provides a centralized, transparent reservation workflow for seminar halls, auditoriums, high-performance computing labs, smart classrooms, and specialized AV equipment.

The Core Non-Negotiable System Invariant

Zero Double-Booking, Structurally Guaranteed: No two approved or pending reservations may ever overlap in time for the same resource, regardless of concurrent network bursts, parallel submissions, or serverless cold starts.

1.1 Layered Architecture Overview



2. Concurrency Control & The Conflict-Free Booking Engine

2.1 The Mathematical Overlap Condition

Two time intervals [S_A, E_A) and [S_B, E_B) overlap if and only if:

$$(S_A < E_B) \ \&\& \ (E_A > S_B)$$

In MongoDB syntax, checking for existing overlapping bookings executes as:

```
const overlappingBookings = await Booking.find({
  resourceId: targetResourceId,
  status: { $in: ["approved", "pending"] },
  startTime: { $lt: requestedEndTime },
  endTime: { $gt: requestedStartTime }
});
```

2.2 TOCTOU Race Condition Mitigation

A naive 'check then insert' architecture suffers from **Time-of-Check to Time-of-Use (TOCTOU)** race conditions when two users submit requests simultaneously. To guarantee atomicity:

- **MongoDB Multi-Document ACID Transactions:** Conflict inspection and record creation execute inside `session.withTransaction()` under Snapshot Isolation.

- **Compound Indexing:** A compound index on `{ resourceId: 1, startTime: 1, endTime: 1, status: 1 }` enforces sub-millisecond query performance and index-level range scans.

2.3 Alternative Time Slot Suggestion Algorithm

When a conflict occurs (HTTP 409 Conflict), the system executes `suggestAlternativeSlots` to calculate open time windows on the requested date:

1. Fetches facility operating bounds (08:00 to 20:00).
2. Pulls all active reservations sorted chronologically.
3. Iterates across intervals to discover available gaps $\geq$ requested duration.
4. Returns the top 3 alternative candidate slots directly inside the 409 error envelope.

3. Security Architecture, Authentication & RBAC

3.1 Role-Based Access Control (RBAC)

Feature / Endpoint	Student	Faculty	Administrator
Browse Facilities & Availability	ALLOWED	ALLOWED	ALLOWED
Submit Booking Request	ALLOWED (Pending)	ALLOWED (Priority)	ALLOWED (Auto-approved)
Cancel Own Reservation	ALLOWED	ALLOWED	ALLOWED
Approve / Reject Other Bookings	403 FORBIDDEN	403 FORBIDDEN	ALLOWED
Resource CRUD (Create/Edit/Delete)	403 FORBIDDEN	403 FORBIDDEN	ALLOWED
AI Natural Language Search	ALLOWED	ALLOWED	ALLOWED

3.2 Dual-Token Silent Refresh Queue Pattern

- **Access Token (15 mins):** Transmitted via `Authorization: Bearer <token>` header. Carries user ID, role, and email.
- **Refresh Token (7 days):** Stored inside a secure `httpOnly` cookie (immune to JavaScript XSS theft).
- **Axios Promise Queue:** When concurrent requests receive 401s, Axios queues all failed requests, executes a single refresh call, attaches the new token, and replays all queued promises seamlessly.

3.3 Institutional Domain Guardrails

- Student & Faculty registrations strictly require `@eec.srmrmp.edu.in` domain emails.
- Administrator role is strictly whitelisted to authorized institutional admin addresses (`keerthanaaruvi2008@gmail.com`).

4. Database Schema Specifications

Model	Key Fields	Indexes & Invariants
User	name , email , passwordHash , role , department , isActive	email (unique, indexed), role (indexed), bcrypt 10 salt rounds pre-save hook.
Resource	name , type , capacity , location (building, floor, room), amenities , status , isActive	Compound index { type: 1, status: 1, isActive: 1 }, { capacity: 1, isActive: 1 }. Soft-delete support.
Booking	resourceId , bookedBy , approvedBy , title , startTime , endTime , status , rejectionReason	Compound index { resourceId: 1, startTime: 1, endTime: 1, status: 1 }. Foreign key references to User & Resource.

5. AI Assistant & Sidecar Microservice

- **Read-Only Microservice:** The Python/FastAPI AI service has zero write access to the database.
- **Structured JSON Output:** It parses natural language into typed query filters (minCapacity , resourceType , date , timeWindow).
- **Deterministic Validation:** The Node.js backend executes deterministic database queries against the structured filters and passes booking actions through bookingConflictService .
- **Graceful Regex Fallback:** If third-party LLM APIs are offline or unconfigured, the system automatically falls back to an internal regex parser with 100% uptime.

6. Tech Expert & Architect Interview Master Guide (Top 15 In-Depth Q&As)

Q1. How does your system guarantee that two users submitting for the exact same slot at the same second cannot double-book?

Answer: Concurrency is handled through a 3-layer defense: (1) The mathematical overlap query `(startTime < reqEnd && endTime > reqStart)` checks active reservations. (2) Writes are wrapped inside MongoDB multi-document ACID transactions with Snapshot Isolation (`session.withTransaction()`), ensuring simultaneous writes fail with a write conflict. (3) Compound indexing on `{ resourceId: 1, startTime: 1, endTime: 1, status: 1 }` ensures atomic lookup efficiency.

Q2. Why did you use Dual-Token (Access + Refresh) authentication instead of storing a JWT in localStorage forever?

Answer: Storing long-lived tokens in localStorage exposes users to complete account takeover via Cross-Site Scripting (XSS). We use a short-lived 15-minute Access Token for API requests, and a 7-day Refresh Token stored inside an `httpOnly`, `Secure`, `SameSite=Lax` cookie that JavaScript cannot access. If an access token is intercepted, the vulnerability window is limited to 15 minutes.

Q3. How does your Axios client handle token expiration without logging out the user or showing errors?

Answer: We implemented a Promise Queue in the Axios response interceptor. When a 401 error is detected, `isRefreshing` is set to true and subsequent requests are pushed into a `failedQueue`. A single call is made to `/api/v1/auth/refresh`. Once a new access token arrives, all queued requests are updated with the new Authorization header and replayed transparently without interrupting the user.

Q4. How does your system operate across serverless platforms (Vercel Lambdas) where RAM memory is destroyed between requests?

Answer: The backend is designed to be 100% stateless: (1) User auth is cryptographically verified via JWTs rather than server RAM session maps. (2) In `server/api/index.js`, the Mongoose connection promise is cached across warm lambda invocations. (3) OTP verification uses stateless HMAC signature validation and format checking. (4) Data is persisted in MongoDB Atlas Cloud.

Q5. Why did you separate the AI service into a FastAPI microservice instead of running everything in Node.js?

Answer: (1) Python has the richest ecosystem for AI/LLM libraries (Pydantic, Google GenAI SDK). (2) Fault isolation: if the LLM provider experiences latency spikes or crashes, the core Express booking portal remains 100% functional. (3) Security boundary: the AI service is strictly read-only and cannot mutate database state directly.

Q6. How do you prevent NoSQL Injection attacks in MongoDB?

Answer: We use a 3-stage defense: (1) Zod schema validation strips or rejects unexpected payload types at the HTTP boundary. (2) The `sanitizeNoSql` middleware recursively removes keys containing `$` or `.` from request bodies, query strings, and params. (3) Mongoose strongly-typed schemas enforce strict casting.

Q7. What is the soft-delete pattern and why is it preferred over hard deletion?

Answer: Instead of `deleteOne()`, we set `isActive: false`. This preserves referential integrity (historical bookings and audit logs still reference valid resource IDs), allows instant recovery from accidental deletion, and maintains analytics data. Indexes filter out inactive documents automatically.

Q8. How would you scale this architecture to handle 1,000,000 users?

Answer: (1) **Redis Caching:** Cache resource catalogs and availability timelines with a 30s TTL. (2) **Redis Distributed Locks (Redlock):** Acquire resource-date locks in Redis before starting database transactions. (3) **Read Replicas:** Route heavy `GET /resources` and `GET /availability` queries to MongoDB Secondary read nodes. (4) **Message Queues (BullMQ / RabbitMQ):** Offload email notifications and AI processing to background worker pools.

Q9. What is the significance of the 409 Conflict status code and how does your UI handle it?

Answer: HTTP 409 Conflict specifically signifies that a requested mutation contradicts current server state. Instead of presenting a generic failure toast, our frontend parses the 409 response envelope to extract the conflicting booking details and renders interactive "Suggested Alternative Slot" pill chips, allowing the user to select an open window in 1 click.

Q10. Why did you use Bcrypt with 10 salt rounds for password hashing?

Answer: Bcrypt is an adaptive key derivation function based on the Blowfish cipher. Salt rounds (cost factor $10 = 2^{10} = 1024$ iterations) introduce calibrated computational delay (~80-100ms per hash), making offline rainbow table and brute-force GPU attacks computationally infeasible while keeping login latency imperceptible to users.